

DAA - Assignment - 2

Name: ANUSKA NATH
Roll: 002311001003
Dept: IT
Year: VU3

Q) Take 3 sorted lists A, B, C

- Write two way merging algorithms for merging the lists into a single sorted list.
- Analyse the number of computations for all possible pairwise merging.
- Discuss the correctness of your algorithms.

Answers

(a) Two way merging algorithms

Since only two lists can be merged at a time, merging three lists requires pairwise merging.

There are 3 possible merging orders.

- 1) Merge A and B, then merge result with C.
- 2) Merge A and C, then merge result with B.
- 3) Merge B and C, then merge result with A.

Algorithm 1: Merge (A, B)

Input: Two sorted lists $A[1 \dots n]$ and $B[1 \dots n]$

Output: A single sorted list D containing all elements of A and B.

Algorithm:

1. $i \leftarrow 1, j \leftarrow 1$
2. create an empty list D
3. while $i \leq \text{length}(A)$ and $j \leq \text{length}(B)$
- ~~4. if $A[i] \leq B[j]$~~
4. if $A[i] \leq B[j]$


```

5.     append A[i] to D
6.     i ← i + 1
7.     else
8.         append B[j] to D
9.         j ← j + 1
10.    while i ≤ length(A)
11.        append A[i] to D
12.        i ← i + 1
13.    while j ≤ length(B)
14.        append B[j] to D
15.        j ← j + 1
16.    return D.

```

Algorithm 2: Merging three sorted lists

1) Case 1: Merge (A+B) first, then C

$D \leftarrow \text{Merge}(A, B)$

$\text{Final} \leftarrow \text{Merge}(D, C)$

2) Case 2: Merge (A+C), then B

$D \leftarrow \text{Merge}(A, C)$

$\text{Final} \leftarrow \text{Merge}(D, B)$

3) ~~Case~~ Case 3: Merge (B+C) first, then A

$D \leftarrow \text{Merge}(B, C)$

$\text{Final} \leftarrow \text{Merge}(D, A)$

(b) Analysis of number of computations.

Let:

i) $|A| = m$

ii) $|B| = n$

iii) $|C| = p$

Cost of Two way merge:

Merging 2 sorted lists of

$m + n - 1$

size x and y requires
comparisons.

1) Case 1: Merge $(A+B)$ first, then C .

i) Merging A and $B \rightarrow (m+n-1)$ comparisons.

ii) Merging $(A+B)$ with $C \rightarrow (m+n+p-1)$ comparisons.

Total comparisons:

$$(m+n-1) + (m+n+p-1) = 2m + 2n + p - 2.$$

2) Case 2: Merge $(A+C)$ first, then B .

i) Merging A and $C \rightarrow (m+p-1)$ comparisons.

ii) Merging $(A+C)$ with $B \rightarrow (m+p+n-1)$ comparisons.

Total comparisons:

$$(m+p-1) + (m+p+n-1) = 2m + n + 2p - 2.$$

3) Case 3: Merge $(B+C)$ first, then A .

i) Merging B and $C \rightarrow (n+p-1)$ comparisons.

ii) Merging $(B+C)$ with $A \rightarrow (n+p+m-1)$ comparisons.

Total comparisons:

$$(n+p-1) + (n+p+m-1) = m + 2n + 2p - 2.$$

Observations:

The total number of comparisons depends on the order of merging.

$\therefore$ To minimise comparisons, the two smallest lists should be merged first.

(c) Correctness of the Algorithm

We prove the correctness of Merge (A, B) algorithm using loop invariant technique.

1) Loop Invariant

At the start of each iteration of the while loop (line 3):

The list contains the smallest elements from $A[1 \dots i-1]$ and $B[1 \dots j-1]$ in sorted order.

2) Initialization

Before the first iteration:

i) $i=1$, $j=1$

ii) D is empty

Thus D initially contains the smallest element of the empty prefixes of A and B . The invariant holds.

3) Maintenance

At each iteration:

i) The smaller of $A[i]$ and $B[j]$ is appended to D

ii) The corresponding index is incremented.

Since the smallest remaining element is always chosen, D remains sorted and correct.

Therefore, the invariant is maintained.

4) Termination

The loop terminates when either A or B is exhausted. The remaining elements of the other list are appended in order.

At termination:

i) All elements of A and B are in D .

ii) D is fully sorted.

Thus, the algorithm is correct.

Correctness of Three List Merge

Since :

i) Each two way merge is correct

ii) The output of the first merge is sorted

The second merge also produces a correctly sorted list.

Hence, merging three sorted lists using pairwise merging is correct.